

Dokumentacja przeprowadzonych testów

Aurumveil: An Optimization System for Resource Allocation and Routing in a Dwarf Kingdom

Cel testów

Celem przeprowadzonych testów była weryfikacja poprawności działania wszystkich zaimplementowanych algorytmów, poprawności integracji poszczególnych modułów systemu oraz stabilności działania aplikacji w różnych scenariuszach użytkowania.

Testy objęły zarówno warstwę algorytmiczną, jak i interfejs użytkownika, mechanizmy przetwarzania danych oraz moduły odpowiedzialne za przechowywanie i kompresję informacji.

Środowisko testowe

Element	Wartość
Język programowania	Python 3.13.13
Silnik algorytmiczny	C++20
Kompilator	GNU G++
System budowania	CMake 4.3.2
Interfejs użytkownika	PySide6 (Qt)
Format danych	JSON

Testy funkcjonalne

Problem maksymalnego przepływu

Test MF-01

Opis testu

Weryfikacja poprawności działania algorytmów Forda–Fulkersona oraz Edmondsa–Karpa odpowiedzialnych za wyznaczenie maksymalnego przepływu w sieci reprezentującej przydział krasnoludków do kopalń.

Dane wejściowe

- Liczba krasnoludków: 1000
- Liczba kopalń: 25
- Maksymalna pojemność pojedynczej kopalni: 100
- Graf przepływowy wygenerowany na podstawie danych świata

Oczekiwany rezultat

Poprawne wyznaczenie maksymalnego przepływu oraz przydzielenie krasnoludków do kopalń bez przekroczenia ograniczeń pojemnościowych.

Uzyskany rezultat

Oba algorytmy poprawnie wyznaczyły maksymalny przepływ dla badanego grafu. Uzyskane wyniki były zgodne i spełniały wszystkie ograniczenia pojemnościowe zdefiniowane w sieci. Wygenerowany przydział krasnoludków do kopalń został zweryfikowany pod kątem poprawności i nie wykryto przypadków naruszenia ograniczeń przepływu.

Status

- ☒ PASS
 - ☐ FAIL
-

Problem minimalnego kosztu

Test MC-01

Opis testu

Weryfikacja poprawności działania algorytmu Min-Cost Max-Flow odpowiedzialnego za wyznaczenie przydziału krasnoludków do kopalń przy jednoczesnej minimalizacji całkowitego kosztu transportu oraz zachowaniu maksymalnego możliwego przepływu.

Dane wejściowe

- Liczba krasnoludków: 1000
- Liczba kopalń: 25
- Koszty transportu: wyznaczone na podstawie odległości pomiędzy krasnoludkami a kopalniami
- Ograniczenia pojemnościowe kopalń zgodne z danymi świata

Oczekiwany rezultat

Wyznaczenie rozwiązania zapewniającego maksymalny przepływ przy jednoczesnym minimalnym koszcie transportu.

Uzyskany rezultat

Algorytm poprawnie wyznaczył maksymalny przepływ w sieci oraz przypisał krasnoludki do kopalń w sposób minimalizujący całkowity koszt transportu. Wszystkie ograniczenia pojemnościowe zostały zachowane, a otrzymane wyniki były zgodne z oczekiwaniami wynikającymi z modelu przepływu minimalnokosztowego. Poprawność wygenerowanych przydziałów została zweryfikowana na podstawie danych wejściowych oraz wyników prezentowanych przez aplikację.

Status

- ☒ PASS
- ☐ FAIL

Problem otoczki wypukłej

Test CH-01

Opis testu

Weryfikacja poprawności wyznaczania otoczki wypukłej dla zbioru punktów reprezentujących położenie kopalń na mapie królestwa. Test miał potwierdzić, że algorytm poprawnie identyfikuje punkty znajdujące się na granicy zbioru oraz pomija punkty położone wewnątrz otoczki.

Dane wejściowe

- Liczba punktów: 50
- Współrzędne: pozycje kopalń zapisane w danych świata

Oczekiwany rezultat

Poprawne wyznaczenie listy punktów należących do otoczki wypukłej oraz zachowanie właściwej kolejności ich występowania na obwodzie figury.

Uzyskany rezultat

Algorytm poprawnie wyznaczył otoczkę wypukłą dla wszystkich punktów wejściowych. Punkty należące do granicy zbioru zostały prawidłowo uwzględnione w wyniku, natomiast punkty znajdujące się wewnątrz obszaru zostały pominięte. Wyniki były zgodne z oczekiwaniami oraz zweryfikowane wizualnie na mapie prezentowanej przez aplikację.

Status

- ☒ PASS
- ☐ FAIL

Problem najgłośniejszego krasnoludka

Test LD-01

Opis testu

Weryfikacja poprawności działania algorytmu wyszukiwania maksimum na przedziale przy wykorzystaniu drzewa przedziałowego (Segment Tree). Wyniki zostały porównane z rezultatami uzyskanymi metodą przeszukiwania liniowego (Brute Force).

Dane wejściowe

- Zbiór danych: 500 obiektów
- Poziomy głośności: 1–250
- Przedział: [100, 350]

Oczekiwany rezultat

Poprawne wskazanie krasnoludka o najwyższym poziomie głośności w zadanym przedziale.

Uzyskany rezultat

Segment Tree poprawnie wskazało krasnoludka o największym poziomie głośności w zadanym przedziale. Wynik był zgodny z rezultatem uzyskanym metodą przeszukiwania liniowego.

Status

- ☒ PASS
 - ☐ FAIL
-

Kompresja danych

Test HC-01

Opis testu

Weryfikacja poprawności działania algorytmu Huffmana.

Dane wejściowe

- Plik stanu świata (world state)
- Dane zawierające informacje o krasnoludkach, kopalniach oraz strażnikach
- Liczba symboli wynikająca z zawartości pliku wejściowego

Oczekiwany rezultat

Poprawna kompresja danych oraz ich późniejsza dekompresja bez utraty informacji. Plik uzyskany po dekompresji powinien być identyczny z plikiem wejściowym.

Uzyskany rezultat

Algorytm Huffmana poprawnie skompresował dane wejściowe oraz odtworzył ich pierwotną postać podczas dekompresji. Zweryfikowano zgodność pliku wynikowego z plikiem źródłowym i nie stwierdzono utraty ani modyfikacji danych.

Nie przeprowadzano szczegółowych testów czasowych ani analiz wydajnościowych. W kontekście realizowanego projektu kluczowym celem było potwierdzenie poprawności działania algorytmu oraz zachowanie integralności danych po procesie kompresji i dekompresji. Czas wykonania operacji nie stanowił istotnego kryterium oceny funkcjonalności systemu.

Status

- ☒ PASS
 - ☐ FAIL
-

Wyszukiwanie danych

Test DS-01

Opis testu

Weryfikacja poprawności działania mechanizmu wyszukiwania wyeksportowanych wyników algorytmów. Test polegał na sprawdzeniu, czy system poprawnie odnajduje zapisane rezultaty na podstawie nazwy pliku zawierającej skrót (hash) danych wejściowych oraz nazw wykorzystanych algorytmów.

Dane wejściowe

- Zbiór wyeksportowanych wyników algorytmów
- Hash danych wejściowych zapisany w nazwie pliku
- Nazwa wybranego algorytmu

Oczekiwany rezultat

Odnalezienie oraz wczytanie poprawnego pliku z wynikami odpowiadającego wskazanemu zestawowi danych wejściowych oraz wybranemu algorytmowi.

Uzyskany rezultat

Program poprawnie wyszukiwał i wczytywał wyeksportowane wyniki na podstawie hasha danych wejściowych oraz nazwy algorytmu zapisanych w nazwie pliku. Dla wszystkich przeprowadzonych przypadków testowych zwrócone zostały właściwe wyniki odpowiadające wskazanym parametrom wyszukiwania.

Status

- ☒ PASS
- ☐ FAIL

Testy interfejsu użytkownika

Funkcjonalność	Wynik
Uruchomienie aplikacji	PASS
Wczytywanie danych	PASS
Uruchamianie algorytmów	PASS
Wyświetlanie wyników	PASS
Obsługa błędnych danych wejściowych	PASS
Kompresja danych	PASS
Wyszukiwanie informacji	PASS

Testy integracyjne

Moduł	Wynik
GUI ↔ Algorytmy	PASS

Moduł	Wynik
GUI ↔ JSON	PASS
Algorytmy ↔ Dane wejściowe	PASS
Kompresja ↔ Magazyn danych	PASS

Podsumowanie wyników

Rodzaj testów	Liczba testów	Zaliczone	Niezaliczone
Funkcjonalne	6	6	0
Integracyjne	4	4	0
Interfejs użytkownika	7	7	0
Łącznie	17	17	0

Wnioski

Przeprowadzone testy potwierdziły poprawność działania wszystkich zaimplementowanych modułów systemu Aurumveil. Uzyskane wyniki są zgodne z założeniami teoretycznymi zastosowanych algorytmów, a integracja poszczególnych komponentów przebiega prawidłowo. Nie stwierdzono błędów wpływających na poprawność działania systemu. Na podstawie przeprowadzonych testów można stwierdzić, że aplikacja spełnia wymagania funkcjonalne określone w specyfikacji projektu.